

Project 3

zkVM-guest API Standardization

Ethereum Foundation · zkVM Breakout Call

github.com/eth-act/zkvm-standards

Accepted Standards

1 RISC-V Target Triple *riscv64im_zicclsm-unknown-none-elf*

Defines a minimal 64-bit RISC-V ISA for proving Ethereum blocks. Uses RV64IM for 64-bit arithmetic efficiency and mandates Zicclsm for transparent misaligned memory access handling.

2 IO Interface *read_input() / write_output()*

Standardizes how guest programs can efficiently read private inputs and write public outputs.

3 Cryptographic Accelerators C Interface *zkvm_accelerators.h*

Defines a portable C API for accessing zkVM-optimized cryptographic operations (based on EVM precompiles), abstracting vendor-specific implementations behind a common header.

Reminder: Implementation Requirements

zkVM vendors are expected to comply with accepted standards.

Migrate to 64-bit RISC-V

The standardized target is RV64IM. Vendors currently running 32-bit zkVMs are expected to plan and execute migration to 64-bit support. This is required for compatibility with the standardized target triple.

Ship a no_std Rust SDK

The target riscv64im-unknown-none-elf is a bare-metal target with no operating system or standard library. Vendor Rust SDKs must be no_std compatible. Any use of std will fail to compile against this target.

Implement the standard IO and accelerator interface

Guest programs must be able to call read_input()/write_output() and access accelerator API as specified. These must be provided by the zkVM runtime and behave per the standard.

Pending Acceptance: Termination Semantics

This standard has been reviewed by many stakeholders. Please move to accept it.

What it specifies

Standardizes what happens when a guest program ends — either successfully or abnormally (panics, aborts, runtime faults). Successful termination may produce a valid proof. Failed termination must be signaled to the host, and its proof must not pass verification.

Security

Prevents buggy or malicious programs from disguising a crash as a successful, provable computation.

Interoperability

Rust, C/C++, and Go all have different failure primitives — this standard gives them a common zkVM-level mapping.

Proof-of-failure

Type 2 verifiers can optionally accept proofs of failed executions with a specific exit code.

Needs Review: Memory Layout Restrictions

Very few vendors have reviewed this standard so far. We encourage all vendors to engage.

What it specifies

Each zkVM vendor may define their own memory layout via vendor-specific linker scripts. Source-code portability is preserved through shared ISA and abstraction layers.

Vendor linker scripts

Each vendor ships their own linker script defining heap, stack, and memory region boundaries. C/C++ and Rust programs must link against it.

No binary portability

An ELF compiled for one zkVM will not run on another. This is an intentional trade-off to give vendors flexibility to optimize proof generation.

Source portability retained

Portable source code is achievable: shared RISC-V ISA, plus standard abstraction layers for IO, accelerators, and termination.

Analogy: embedded systems

Just as RISC-V microcontrollers from different vendors need different linker scripts despite sharing the same ISA, so do zkVMs.